

1 EXERCISE : LIST TRANSFORMATION

In this exercise you will learn how to transform python lists and filter specific elements from them.

1.1 Transform an interval of numbers

1.1.1 Square elements of a list

Create two lists, one is called `interval` and has all integer numbers from 0 to 20 (inclusive) in it, the other is called `sq_interval` and is empty. Use a `for` loop to iterate over the elements of `interval` and fill `sq_interval` with the squared value of each element.

Expected output:

```
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81, 100, 121, 144, 169, 196, 225, 256, 289, 324, 361, 400]
```

Hint: make use of `range()` for creating `interval` and use the `**` operator for squaring.

1.1.2 Remove odd numbers from the squared list

Create another empty list `even_sq_interval`. Fill it with only the **even** numbers from the list `sq_interval`.

Expected output:

```
[0, 4, 16, 36, 64, 100, 144, 196, 256, 324, 400]
```

Hint: remember the `if` statement and combine it with the modulus operator `% 2` in order to estimate oddity/evenness of a number.

1.2 Transform a corrupted string

You are given the badly formatted DNA sequence

```
some_dna = "aACTa TtCcC acCtc\tcaTCC CGGCc\nTaTaT CTGaa"
```

and want to convert it into a more readable version.

1.2.1 Remove white space and convert sequences to capital letters

Copy the given string from this exercise sheet and assign it to a variable named `some_dna` within your python script. Create a list `dna_pieces` that includes all pieces of DNA from within `some_dna`, without white spaces (`' '`, `\n` and `\t`).

Expected output:

```
["aACTa", "TtCcC", "acCtc", "caTCC", "CGGCc", "TaTaT", "CTGaa"]
```

Hint: use `str.split()` (see python cheat sheet on page 1, string) to split the string into a list of pieces.

1.2.2 Convert the pieces into capital letters and combine them into a multi-line string

Now do a `for` loop over the previously created list `dna_pieces`, convert all letters in each piece into capital letters, and add them to the new list `upper_dna_pieces`. Then convert this new list to a string in which all pieces are connected by newline characters (`\n`).

Expected output:

```
AACTA
TTCCC
ACCTC
CATCC
CGGCC
TATAT
CTGAA
```

Hint: the string functions `str.upper()` and `"\n".join()` are useful for this task. Refer to the cheat sheet to see how they are used.

2 EXERCISE : NUCLEOTIDE COUNTING

Your professor has a `.txt` file with a DNA sequence (`strange_dna.txt`) and would like to know which of the nucleotides A, T, G, C can be found how often in that sequence. Unfortunately he doesn't have time for counting and would like you to write the function `count_nucleotides` to make his life easier.

2.1 Create the function

Create the function `count_nucleotides`. Make sure it can be called from another cell with the name of the `.txt` file as its first argument. To begin with, try to make the script print the arguments it was called with.

Expected output: `"files/exercise_2/strange_dna.txt"`

2.2 Read the `.txt` file

Add the `open()` command and a `for` loop to the script in order to load the text from the file into the variable `DNA` within python. `print DNA` to show its contents and make sure it doesn't contain newline characters (`\n`).

Expected output:

```
"TACGGGGCCGACATCTGCGGGGGGGCGGGGCGGCCCCCGGTCAGACC
AGAAAGGAGGCACCCCGCTTCCGCCGTGACTGGCATGTCGGGAGCGGGC
CCCGAGCCAGGCCGCTGAGGGGGCCGCCCTGCGCGCGACCCGCAGGGGC
CCGCGGCGCCGACCAGCCGCCCCGCCAGCCCCCTCGCGCTTCGACGGT
CCGCAACGCGCCAAGCCTCGCGAGCGCCGGCGCCTCCGACCGCGGGTGA
GACCGCGGCGCGCGTAACGCGCTCCAGTGCCAGGCGCGTCCCCCTTACGCC
CGCGGGCAGTCCCCGGGCAGACACACGGAGGACCGGGATCAAATTGAACC
CAGAGCATCACGACACCACGGCCGGGGCCCCGCAACCCGGGGTGGGAGC
GGGACAGA"
```

Hint: open the file in read mode ('r'). Then read it line by line and add the lines to DNA using the += operator. Check the python cheat sheet (page 1, string) for a function to remove newline characters from strings.

2.3 Scan the sequence and count nucleotides

Now add another loop that visits each nucleotide in the variable DNA and counts for all types of nucleotides separately, how often they have been seen. If you use a dictionary for this, print it to the screen.

Expected output: {"A": 62, "T": 31, "C": 159, "G": 148}

Hint: use either 4 separate variables or a dict to keep track of the counts. You can use if/elif/else statements to distinguish between the different nucleotides in each step of the loop (but there is an easier way if you use a dict).

2.4 Report the results

Show the final nucleotides on the screen in an easily readable way.

Expected output:

```
Count of A : 62
Count of T : 31
Count of G : 148
Count of C : 159
```

Hint: You can get all key-value pairs from a dictionary with the function dict.items() and then nicely display them with another for loop and print statements. Alternatively you loop through all the keys directly: for key in dict:.

3 EXERCISE : K-MEANS

OPTIONAL CHALLENGE

The file `data.csv` contains activation maxima extracted from an MRI scan. Each row represents a peak of neural activity, described by its `x` and `y` coordinates in a 2D projection of the brain image.

The goal is to cluster these points to identify distinct regions within the MRI scan.

To achieve this, you will use the k-means clustering algorithm, a widely used method for partitioning data into distinct groups, or clusters. Additionally, you will evaluate the quality of the clustering for `k` between 2 and 6 using the silhouette score, a metric that measures how well-separated the clusters are.

3.1 The k-means clustering algorithm

The k-means clustering algorithm works as follows:

1. **Initialization:** Randomly select `k` initial centroids, which are points representing the center of a cluster, within the bounds of the data.
2. **Assignment:** Assign each data point to the nearest centroid, forming `k` clusters. The nearest centroid is determined using the Euclidean distance:

$$d(p, c) = \sqrt{(p_x - c_x)^2 + (p_y - c_y)^2}$$

3. **Update:** Recalculate the centroids as the mean position of all points assigned to their cluster.
4. **Iteration:** Repeat steps 2 and 3 for a number of iterations.

The final result is a set of `k` clusters, each represented by a centroid. This is illustrated for `k=3` and 15 iterations in Figure 1.

3.2 Task: k-means clustering for k=3

Write a program to implement the k-means clustering algorithm based on the dataset in `data.csv`, using `k=3` and a total of 15 iterations. To initialize the centroids, use the function `create_initial_centroids()` provided below.

Expected output:

```
10.22 13.22
10.45 0.12
-0.41 6.50
```

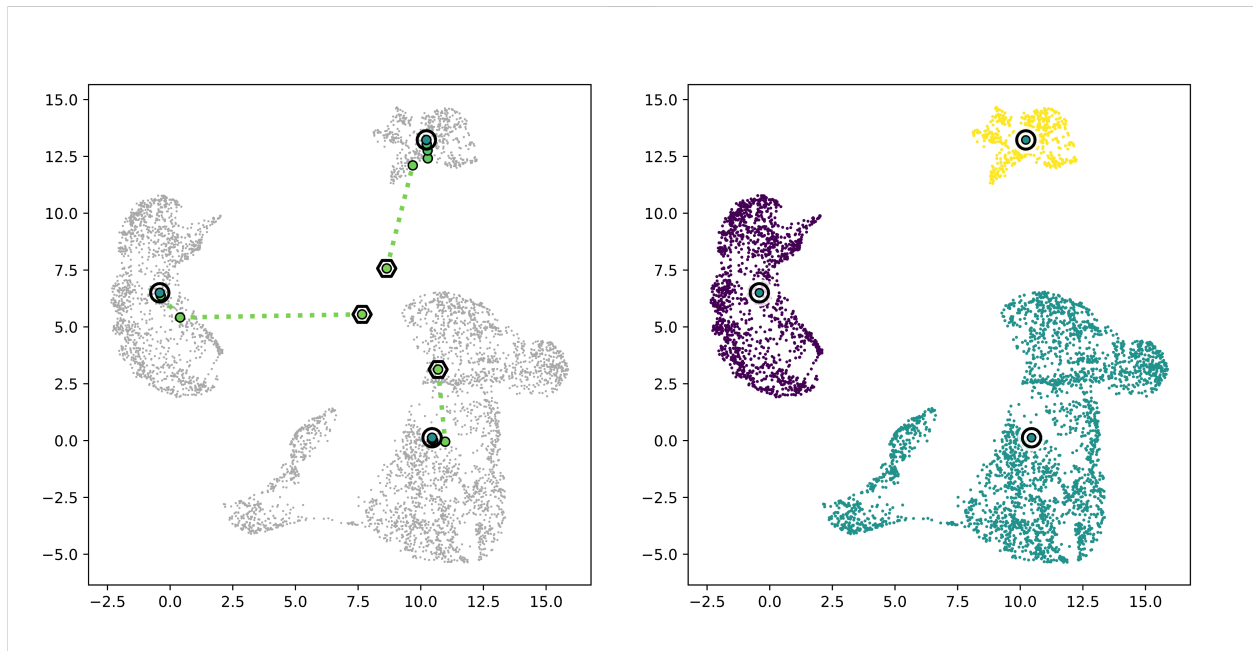


Figure 1: K-Means Clustering for $k=3$. The left panel illustrates the data points, shown as grey dots, initial centroids, shown as hexagons, and the iterative process of updating the centroids, shown as round circles, during the 15 iterations of the k-means algorithm. The right panel shows the final clustering result, with data points grouped into three distinct clusters and centroids marked as dark circles.

```
import numpy as np

def create_initial_centroids(data, k):
    """
    Initializes k centroids randomly within the bounds of the data.
    Parameters:
        data (numpy.ndarray): Array of shape (n_samples, 2)
                             representing the data points.
        k (int): Number of clusters.
    Returns:
        numpy.ndarray: Array of shape (k, 2) with the initial centroids.
    """
    np.random.seed(5)
    centroids = np.zeros((k, 2))
    centroids[:, 0] = np.random.uniform(np.min(data[:, 0]), np.max(data[:, 0]), size=k)
    centroids[:, 1] = np.random.uniform(np.min(data[:, 1]), np.max(data[:, 1]), size=k)
    return centroids
```